

Advanced Lesson 14 - Exercises: Dynamic Interaction, Files and Strings Mikey Ferguson MATH 7013 - Advanced Mathematical Modeling

1: Manipulate - dynamic 2-D and 3-D plots

Exercises 1 and 2 repeat Lesson 4's Exercises 14 and 15 verbatim. The cells below build correctly under a plain kernel, but the sliders are live only inside a Wolfram front end. Evaluated through Wolfram Engine under Jupyter they render as a static snapshot at the initial parameter values, so these cells are left unevaluated here.

```
Manipulate[Plot[E^(-a x), {x, 0, 1}], {a, 0, 10}]
```

Part 2. The plot jumps as the slider moves because the vertical range is recomputed for each value of a . Setting `PlotRange -> All` - or to an explicit interval - pins the axes so the curve moves against a fixed frame.

```
Manipulate[Plot[E^(-a x), {x, 0, 1}, PlotRange -> All], {a, 0, 10}]
```

```
Manipulate[  
  Plot3D[E^(-a x - b y), {x, 0, 1}, {y, 0, 1}, PlotRange -> All],  
  {a, 0, 10}, {b, 0, 10, 2}]
```

Part 3. A control written `{a, 0, 10}` is continuous. Adding a step, as in `{b, 0, 10, 2}`, makes it discrete: the slider snaps to 0, 2, 4, 6, 8, 10.

2: A dynamic algebraic calculator

```
Manipulate[Expand[(x + y)^n], {n, 0, 10, 1}]
```

The step of 1 restricts n to integers, which is the only thing Expand can usefully do with this exponent. Dragging the slider walks up the binomial expansions one row of Pascal's triangle at a time.

3: Writing a list to disk, reading it back

```
Clear[dataList, myData, file];
dataList = {1, 2, 3, 4, 5, 6, 7};
file = FileNameJoin[{$TemporaryDirectory, "dataList.txt"}];
Export[file, dataList, "List"];
file
```

Part 1. Export writes the list to a text file, one entry per line, using the "List" format. The destination here is a file in the session's temporary directory, chosen so that evaluating this notebook does not scatter files into whatever directory happens to be current; substitute any path. The older idiom is Save or Put, which write Wolfram Language syntax rather than plain lines. Export with an explicit format is the more predictable choice, because what comes back is determined by the format rather than by what the expression happened to look like.

```
FilePrint[file]
```

Part 2. FilePrint dumps the raw contents of the file, which is seven lines each holding one integer. Nothing else - no braces, no commas, no header.

```
myData = Import[file, "List"];
myData
```

```
{Head[myData], Length[myData], myData === dataList}
```

Part 3. Import with the same format reads the lines back into a list of integers. The last cell confirms the round trip: the head is List, the length is 7, and the recovered list is identical to the original.

ReadList with the Number type does the same job and is the idiom the lesson was originally written against; Import is the modern spelling.

4: Turning text into a list of characters

```
Characters["Mathematica"]
```

```
Length[Characters["Mathematica"]]
```

Characters splits a string into a list of one-character strings, so the result is {"M", "a", "t", ...} with eleven entries. Note that each entry is itself a string, not a character type - Mathematica has no separate character

data type. This is worth connecting back to Lesson 7: a string is an atom, Length of it is 0, and Characters is the function that converts that atom into a genuine list with a genuine length.